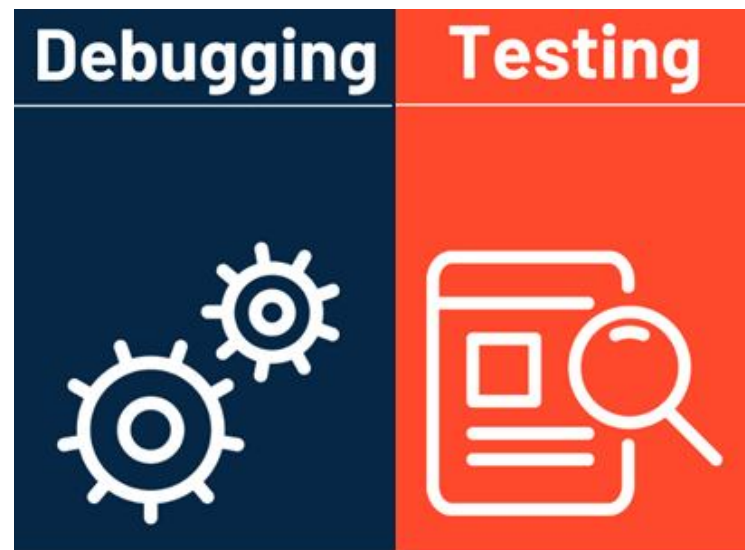




Structured Programming Concepts ***(26CSE0105)***



Program Debugging, Testing & Documentation

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Program Debugging and Testing:***
 - ***Compilation Errors***
 - ***Runtime Errors***
 - ***Logical Errors***
 - ***Debugging Techniques***
 - ***Testing Strategies***
- ***Documentation:***
 - ***Code Readability***
 - ***Code Optimization Basics***

Program Debugging

- **Debugging** is the process of **finding**, **understanding**, and **fixing errors** (**bugs**) in a program so that the program will **produce** the **expected result**.



"A bug-free program runs correctly and gives the expected output."

Program Debugging

Steps in Debugging a Program

- 1 Identify the Problem**
Observe the incorrect output or error message.
- 2 Reproduce the Error**
Run the program again to see the same error.
- 3 Locate the Error**
Use error messages, print statements or debugger.
- 4 Fix the Error**
Correct the code.
- 5 Test Again**
Run the program to verify the fix.
- 6 Repeat if Needed**
Some bugs may lead to new issues.

Why is Debugging Important?

- ✓ Helps in producing correct and reliable programs.
- ✓ Improves program quality and performance.
- ✓ Develops problem-solving and logical thinking skills.
- ✓ Essential in real-world software development.
- ✓ Saves time and effort in the long run.

Types of Bugs

- A **bug** is an **error**, **flaw**, or **defect** in a program that causes it to **behave unexpectedly** or produce **incorrect results**.

Common Types of Bugs in C

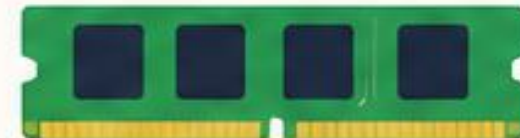
Find Bugs
Fix Bugs
Write Better Code



1. Memory Leaks

Allocating memory but forgetting to release it. The memory remains allocated even when it is no longer needed, leading to wasted memory.

```
#include <stdlib.h>
void example()
{
    int *p = (int *)malloc(100 * sizeof(int));
    // use memory
    // missing free(p);
}
```



Memory allocated
but not released



Memory Leak



Fix: Always release dynamically allocated memory using **free()**.

Common Types of Bugs in C

Find Bugs
Fix Bugs
Write Better Code



2. Buffer Overflows & Out-of-Bounds Access

Writing data beyond the boundary of an array. This can overwrite other memory locations and cause unexpected behavior or security vulnerabilities.

```
#include <stdio.h>
int main()
{
    int arr[5];
    for (int i = 0; i <= 5; i++) // should be i < 5
        arr[i] = i;             // arr[5] is out of bounds
    return 0;
}
```



Out-of-bounds write
(overwrites memory)



Fix: Always check array bounds (use index < size).

Common Types of Bugs in C

Find Bugs
Fix Bugs
Write Better Code



3. Pointer Errors

Using a null pointer (pointer not pointing to a valid memory location) or using dangling references (pointer to memory that has been freed).

Null Pointer Dereferencing

```
#include <stdio.h>
int main()
{
    int *p = NULL;
    printf("%d", *p); // runtime error
    return 0;
}
```

Dangling Pointer

```
#include <stdio.h>
int main()
{
    int *p = (int *)malloc(sizeof(int));
    free(p); // memory freed
    printf("%d", *p); // dangling pointer
    return 0;
}
```



Fix: Check for **NULL** before using pointers and avoid using pointers after freeing memory.

Common Types of Bugs in C

Find Bugs
Fix Bugs
Write Better Code



4. Logical Errors

The program runs without crashing but produces incorrect results due to wrong logic, conditions, or mathematical errors (e.g., division by zero).

Incorrect Condition

```
#include <stdio.h>
int main()
{
    int age = 20;
    if (age = 18)           // should be ==
        printf("Adult");
    else
        printf("Minor");
    return 0;
}
```

Division by Zero

```
#include <stdio.h>
int main()
{
    int a = 10, b = 0;
    int c = a / b;         // runtime error
    printf("%d", c);
    return 0;
}
```



Fix: Use correct logic, check conditions carefully, and avoid invalid operations like division by zero.

Types of Errors



- During **debugging**, programmers generally encounter **three major types of errors**:

Common Types of Errors in C

Type of Error	Description	Example
Syntax Errors	Violation of C language rules. Detected by compiler.	<code>printf("Hello") // missing ; int a = 10</code>
Runtime Errors	Occur during program execution.	<code>int x = 10/0; // division by zero</code>
Logical Errors	Program runs but gives incorrect output.	<code>float area = 2 * 3; // wrong formula for area of circle</code>

Key Difference

- **Syntax / Runtime / Logical Errors** → classify errors based on **when** and **how** they appear in the program.
- **Memory Leak / Buffer Overflow / Pointer Errors** → describe **specific types** (categories) of programming bugs.

Common Debugging Techniques

- Before you start **debugging**, make sure the code is **clean** and **organized**.

1. Read and Understand Error Messages:

- **Compiler** often tell you **what went wrong** and **where**.

```
#include <stdio.h>
int main()
{
    printf("Hello World") // Missing ;
    return 0;
}
```

Compilation failed due to following error(s).

```
main.c: In function 'main':
main.c:5:26: error: expected ';' before 'return'
   5 |     printf("Hello World") // Missing ;
     |                               ^
     |                               ;
   6 |     return 0;
```

Common Debugging Techniques

2. Print Debugging

- Use **printf()** to print values at **different points** in your code to see **what's going wrong**.

```
#include <stdio.h>
int main()
{
    int x = 10, y = 0, z;
    printf("Before division\n"); // Debug output
    z = x / y; // Crashes (division by zero)
    printf("After division\n"); // Never run
    return 0;
}
```

```
Before division
```

```
...Program finished with exit code 136
Press ENTER to exit console.
```

Common Debugging Techniques

3. Trace the Program Manually:

- **Execute** the program **line by line** on paper using **sample input**.

```
#include <stdio.h>
int main()
{
    int a = 5, b = 10, c;
    c = a + b * 2;
    printf("%d\n", c);
    return 0;
}
```

Trace:

$$b \times 2 = 20 \rightarrow a + 20 = 25 \rightarrow c = 25$$

- This is particularly useful for **finding logical errors**.

4. Check Memory Using Debugging Tools:

Tools such as **VALGRIND** can help detect:

- **Memory leaks**
- **Invalid memory access**
- Use of **freed memory**
- especially important when using **pointers** and **dynamic memory**.

Debugging Advantages

- ***Improves software quality***
- ***Reduces program failures***
- ***Increases reliability***
- ***Saves maintenance cost***
- ***Makes programs easier to maintain***
- ***Enhances user satisfaction***

Program Testing

- **Program Testing** is the process of **executing a program** with the purpose of **finding errors (bugs)** and **verifying** that the software works according to the specified requirements.



Objectives of Program Testing:

- **Detect errors and defects in the program.**
- **Verify that the program meets user requirements.**
- **Ensure the program produces the correct output.**
- **Improve software quality and reliability.**
- **Increase user confidence in the software.**

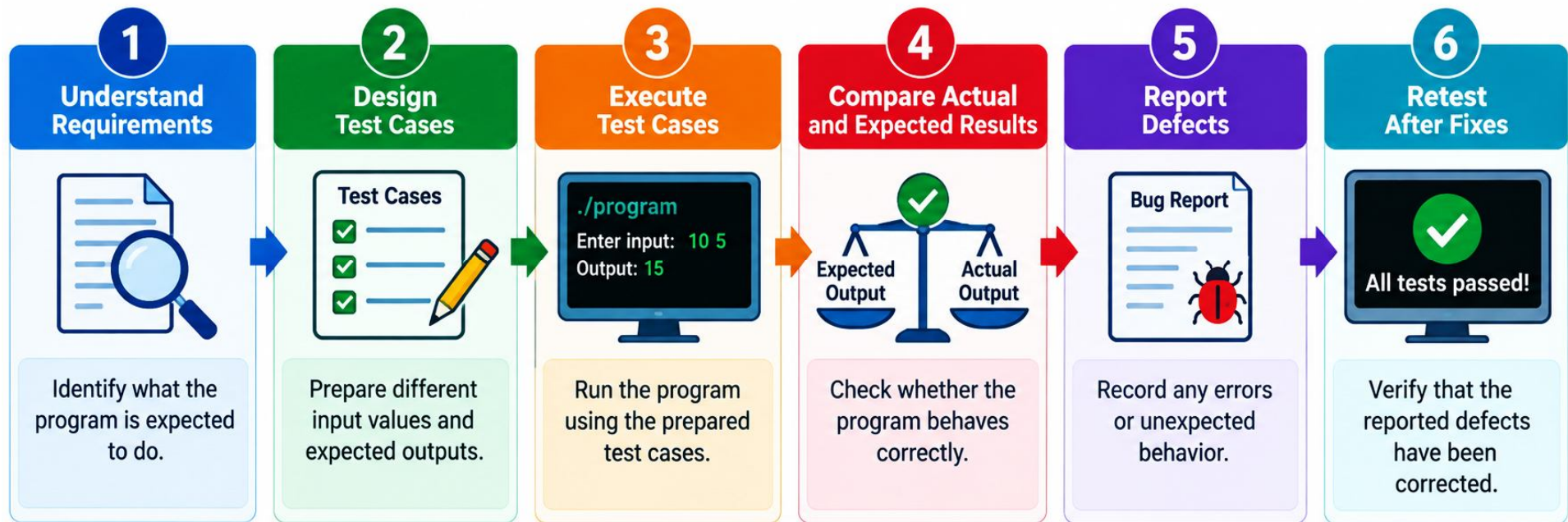
Program Testing



Testing is important because it helps to:

- 1 Find errors and bugs** before the program is used.
Helps detect mistakes early, saving time and cost.
- 2 Verify correctness** of program output.
Ensures the program produces the expected and correct results.
- 3 Check boundary conditions** such as zero, negative values, and maximum values.
Helps find errors at the limits of input values.
- 4 Improve reliability** and reduce unexpected program failures.
Builds trust in the software and ensures smooth execution.
- 5 Validate user input** and handle invalid data properly.
Prevents the program from crashing or giving wrong results due to incorrect input.
- 6 Check different execution scenarios.**
Ensures the program works correctly under all possible situations.

Program Testing Process



Program Testing Techniques



1 Unit Testing

- Tests individual functions or modules independently.
- Usually performed by developers.

Example:

Testing a function that calculates the factorial of a number.

```
int factorial(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * factorial(n - 1);  
}
```

Test Case:

Input: 5

Expected Output: 120

Actual Output: 120 ✓

2 Integration Testing

- Tests the interaction between two or more modules.
- Ensures that combined modules work correctly.

Example:

Checking whether the Login module correctly communicates with the Database module.



- ✓ Valid user sends credentials to the database
- ✓ Database verifies credentials
- ✓ Login success or failure is returned

Program Testing Techniques



3 System Testing

- Tests the complete software as a whole.
- Verifies that all system requirements are satisfied.

Example:

Testing an entire Student Management System.



- ✓ All features work correctly
- ✓ System is stable and reliable
- ✓ Meets all functional and non-functional requirements

Validates the complete system before release.

4 Acceptance Testing

- Performed by the customer or end user.
- Confirms that the software meets business requirements before deployment.

Example:

A college verifies that the attendance management software fulfills all its requirements before using it.



- ✓ Meets requirements
- ✓ Easy to use
- ✓ Works as expected
- ✓ Ready for deployment



Ensures the software is ready for real-world use and satisfies user needs.

Program Testing Techniques

```
#include <stdio.h>
int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    printf("Sum = %d", a + b);
    return 0;
}
```

Example Test Cases (for addition program)

Test Case	Input (a, b)	Expected Output	Purpose
1	5, 3	Sum = 8	Normal case
2	0, 0	Sum = 0	Boundary case
3	-5, 10	Sum = 5	Negative input
4	1000, 2000	Sum = 3000	Large input
5	a, 5	Invalid input	Non-numeric input



Good Practices

- ✓ Read and understand error messages.
- ✓ Use meaningful variable names.
- ✓ Use proper indentation.
- ✓ Test with different inputs.
- ✓ Keep the code simple and modular.
- ✓ Use a debugger (e.g., GDB) or print statements.



Key Takeaway:

Debugging removes errors, and testing builds confidence.
A well-debugged and well-tested program is reliable, efficient, and ready for real-world use.

- **Program documentation** is the **process** of **writing clear information** about a **C program** so that its;
 - ❖ **Purpose**
 - ❖ **Logic**
 - ❖ **Structure**
 - ❖ **Usage** and
 - ❖ **Maintenance**
- can be **easily understood** by **programmers** and **users**.

Documentation in C



Why is Documentation Important?

Good documentation makes your code understandable today, maintainable tomorrow, and useful for others.

Document
Today
Build a Better
Tomorrow!

1 Helps understand the purpose and working of the program.



```
// This program calculates  
the area of a circle  
  
area = 3.14 * r * r;
```

Documentation explains what the program does and how it works.

2 Makes code easier to read and maintain.

Without Documentation

```
a=0;  
b=10;  
for(i=0;i<10;i++){  
    sum=sum+a[i];  
}
```



With Documentation

```
int i, sum = 0;  
// Loop through 10 elements  
for (i = 0; i < 10; i++) {  
    sum = sum + a[i];  
}
```



Well-documented code is easier to read, understand, and maintain.

3 Helps programmers debug and modify the program later.



Documentation

- ✓ Program logic
- ✓ Function details
- ✓ Variable meaning
- ✓ Input/Output info

Documentation helps you quickly locate issues and make changes.

4 Makes it easier for another programmer to understand and reuse the code.

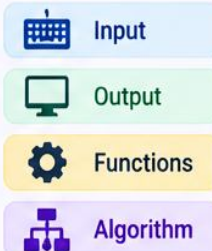


Good documentation helps other programmers understand and reuse your code easily.

5 Provides information about inputs, outputs, functions, and algorithms.

Program Documentation

- Purpose
- Input and output
- Function details
- Algorithm / Logic
- Examples



Documentation provides complete details about inputs, outputs, functions, and algorithms.

6 Useful for future maintenance and upgrades.



- ✓ Easier updates
- ✓ Quick modifications
- ✓ Long-term usability
- ✓ Supports future development

Well-written documentation is valuable for future maintenance and upgrades.

Documentation in C

1. Internal Documentation

Internal documentation is written inside the source code.



Well
Documented
Code
is Better Code!

What is Internal Documentation?

Internal documentation is written **inside the source code**. It helps the programmer (and others) to understand the purpose, logic and working of the code.

It mainly consists of:



Comments

Explain what the code does and why.



Meaningful variable names

Use descriptive names instead of single letters (e.g., marks instead of m).



Function descriptions

Add comments to explain the purpose, inputs/outputs and behavior of functions.

Documentation in C



1. Internal Documentation

Internal documentation is written inside the source code.



Well
Documented
Code
is Better Code!

Example: Internal Documentation in C

student_marks.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Store student's marks
6      int marks = 85;
7
8      // Display marks
9      printf("Marks = %d", marks);
10
11     return 0;
12 }
```

Header file inclusion
(self-explanatory)

Comment
(explains the purpose)

Meaningful variable name
(marks instead of m)

Comment
(explains the purpose)

Function description
(built-in function with purpose)

Indicates successful
program termination

Documentation in C



2. External Documentation

External documentation is written **outside the program code**.

Examples include:



User Manual

Guides end users on how to use the software.



Installation Guide

Explains how to install and configure the software.



Design Documents

Describes the design and architecture of the software.



Technical Reports

Provides detailed reports on development, testing and results.



API Documentation

Describes the functions, classes or APIs provided by the software.

Purpose:

- Helps users install, operate, and maintain the software.
- Useful for different stakeholders (users, developers, administrators).
- Makes the software easy to understand and use.

3. Technical Documentation

Provides detailed information for **programmers and developers**.

Includes:



Program structure

Overall structure of the program.



Functions and modules

Details of each function/module and their purpose.



Variables and data structures

Description of variables, arrays, structures, etc.



Algorithms

Logic and step-by-step approach.



Dependencies

Libraries, tools or external components used.



Technical design

Detailed design of system components.

Purpose:

- ✓ Helps developers understand the internal working of the program.
- ✓ Useful for development, debugging and future modifications.
- ✓ Acts as a reference for new team members.

Documentation in C



4. Testing Documentation

Records information about **testing performed on the program**.

Includes:



Test cases

Different scenarios to test.



Input values

Test data used as input.



Expected output

Correct output for each test case.



Actual output

Output produced by the program.



Test results

Pass / Fail status.



Defects found

List of errors or unexpected behaviour.

Test Case	Input	Expected Output	Actual Output	Result
TC1	5	5	5	Pass
TC2	10	20	20	Pass
TC3	-1	Error	Error	Pass
TC4	a	Error	Error	Pass

Purpose:

- ✓ Ensures the software works correctly.
- ✓ Helps identify and track defects.
- ✓ Useful for verification and quality assurance.
- ✓ Provides a record for future reference.

5. Maintenance Documentation

Helps developers **modify, update, and maintain** the program in the future.

Includes:



Changes made

Details of modifications done.



Bugs fixed

Description of bugs and how they were resolved.



Version history

Records of different versions and release information.



Modified functions

List of updated or new functions.



Known issues

Known limitations or unresolved problems.



Future improvements

Suggestions for future updates and enhancements.

Purpose:

- ✓ Helps in long-term maintenance of the software.
- ✓ Useful when updates or new features are added.
- ✓ Helps new developers understand previous changes.
- ✓ Improves the reliability and lifespan of the software.

Code Optimization Basics

- **Code optimization** is the **process** of **improving a program** so that it uses **less time, memory, and system resources** while **producing the same correct output**.

Same Output + Less Time + Less Memory = Optimized Code

Code Optimization Basics



Why is **Code Optimization** Important?

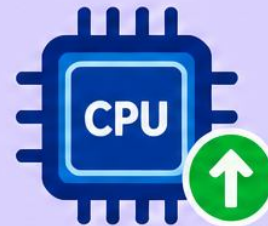
Optimization can help to:



 Reduce execution time



 Reduce memory usage



 Improve CPU utilization



 Reduce power consumption



 Improve program performance



 Make programs more efficient



 Improve scalability for large inputs

Compilation Errors

- **Compilation Error** is an **error** that occurs **during the compilation** of a program, before **it is executed**.

 Common compilation errors are;		
1	Missing semicolon (;)	<code>int a = 10</code> 
2	Syntax errors	<code>if (a > 0)</code> 
3	Undeclared variables	<code>printf("%d", x);</code> 
4	Missing or incorrect header files	<code>#include <stdio.h></code> 
5	Missing braces ({ })	<code>if (a > 0) printf("Hi");</code> 
6	Incorrect function declaration or definition	<code>int add(int a, int b)</code> 
7	Incorrect data type usage	<code>int name = "Raj";</code> 
8	Misspelled keywords or identifiers	<code>printrf("Hello");</code> 

Compilation Errors



Tips to avoid compilation errors;

1

End every statement with a semicolon (;).



2

Declare variables before using them.

```
int x;
```

3

Include all required header files.



4

Check matching braces and parentheses.

{ } ()

5

Use meaningful variable names.

```
int count;
```

6

Use proper indentation to improve readability.



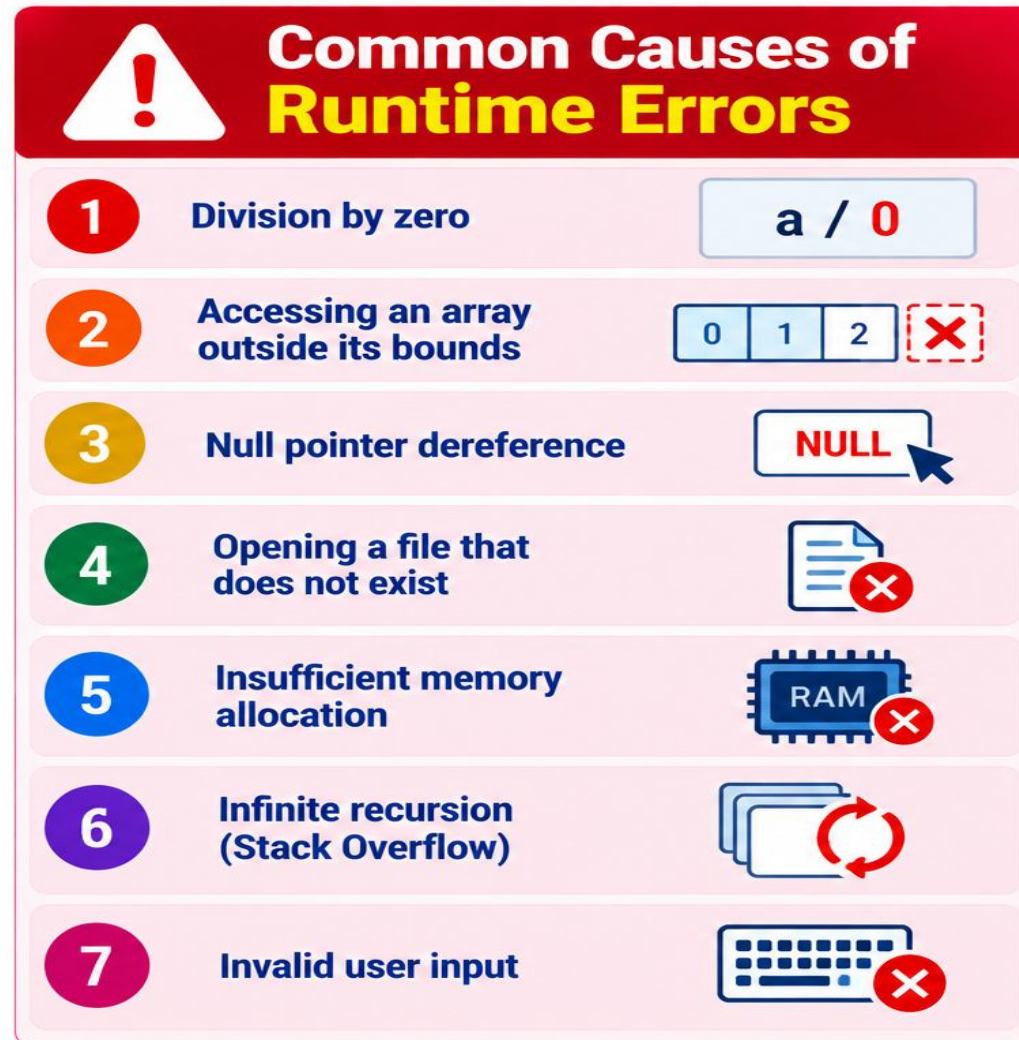
7

Read compiler messages carefully.



Run Time Errors

- **Runtime Error** is an error that occurs **while a program is executing (running)**, even though it has been **compiled successfully**.



Run Time Errors



Tips to Prevent Runtime Errors

1

Validate user input before processing.



2

Check for division by zero.



3

Ensure arrays are accessed within their bounds.



4

Verify pointers are not NULL before using them.



5

Check whether files open successfully before reading or writing.



6

Test the program with different inputs and edge cases.



Logical Errors

- **Logical Error** is an error **in the logic** of a program and it **compiles successfully** and runs **without crashing**.
- But it produces **incorrect** or **unexpected output** because the **logic used** in program is **wrong**.



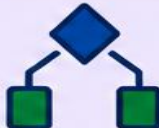
Common causes of Logical errors:

1	Using the wrong arithmetic operator (+ instead of *)	<pre>int area = a + b;</pre>	✗
2	Incorrect formula or algorithm	<pre>int sum = n * (n - 1);</pre>	✗
3	Wrong comparison operator (> instead of >=)	<pre>if (age > 18)</pre>	✗
4	Incorrect conditional statements	<pre>if (x = 5)</pre>	✗
5	Wrong loop condition	<pre>for (i = 0; i > 10; i++)</pre>	✗
6	Incorrect variable initialization	<pre>int count; // not initialized</pre>	✗
7	Using the wrong variable in calculations	<pre>int avg = total / n; // intended: sum / n</pre>	✗

Logical Errors



Tips to prevent logical errors:

- 1** Use the correct arithmetic operator as per the requirement.  
- 2** Verify the formula or algorithm before implementation.  
- 3** Use the correct comparison operator.  
- 4** Write correct conditional statements and check all cases (true and false).  
- 5** Use the correct loop condition and verify the start, end and update values.  
- 6** Initialize variables with appropriate values before using them.  
- 7** Use the correct variable in calculations and double-check the logic.  

Errors Comparison

Comparison of Errors in C Programming

Feature	Compilation Error	Runtime Error	Logical Error
Occurs	During compilation	During execution	During execution
Detected by	Compiler	Runtime environment	Programmer / Tester
Program Compiles?	✗ No	✓ Yes	✓ Yes
Program Runs?	✗ No	✗ May crash / stop	✓ Yes
Output	No output	Program stops unexpectedly	Incorrect output
Common Cause	Syntax mistakes	Illegal operations	Wrong logic or formula
Example	Missing ;	Division by zero	Using + instead of *
Solution	Correct syntax errors and recompile	Handle runtime conditions	Correct the program logic

Quick Examples

Compilation Error	Runtime Error	Logical Error
<code>int a = 10</code> ✗ (Missing ;)	<code>int result = 10 / 0;</code> ✗ (Division by zero)	<code>area = length + breadth;</code> ✗ (Should be length * breadth)



Thank You!

